TCP vs UDP Performance Comparison

CMPE-206 Computer Network Design Project

Prepared for:

Yalda Edalat, Ph.D., Computer engineering instructor at San José State University

Prepared by:

Bhimsen Thapa Chhetri, Software Engineering, MS at SJSU

John Ian Buñag, Software Engineering, MS at SJSU

May 10, 2026

Abstract

Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) are the two primary transport-layer protocols used in IP networks. They both are used for the transmission of data between the endpoints, but both of them have their own significance. They both differ on the connection type, reliability, performance, speed and data size. The selection between TCP and UDP varies between the requirement and nature of data transfer. The purpose of this paper is to perform a performance comparison between TCP and UDP. Using NS-3 the throughput, packet loss, retransmission and fairness will be evaluated between TCP and UDP.

Introduction

Transmission Control Protocol (TCP) is a connection-oriented protocol that is used to ensure the reliable data delivery. It sends data in a sequence without loss. It creates a stable communication by establishing a three way handshake.

User datagram protocol (UDP) is a connectionless transport protocol. It provides a way for applications to send encapsulated IP datagrams without having to establish a connection.

	TCP	UDP
Connection Type	Connection-oriented (3 way handshake)	Connectionless
Sequencing	Yes	No
Data size	Arbitrary length content	Limited message size
Reliability	Reliable (Guaranteed delivery)	Unreliable
Speed	Slower	Faster
Usage	Downloads, File transfer, Web browsing, Email	VOIP, Video, Gaming

Figure 1: Comparison between TCP and UDP

Related Work

To gather more knowledge and understanding on this topic we have reviewed a few related works.

Survey on Transport Layer Protocols: TCP and UDP [2] demonstrates the function of TCP and UDP. The comparison between TCP and UDP on the basics of data transfer features, operations and applications are detailed in the paper. It also presents the use of TCP and UDP on the basis of reliability and speed.

From TCP and UDP performance for Internet over optical packet-switched networks [4] we were able to understand how TCP and UDP perform with OPS networks. The OPS networks have extremely small or no true buffers. It requires fast scheduling and contention resolution. It uses weighted fair queuing which supports QOS and fairness. The packet aggregation and small ingress buffer reduces jitter and significantly boosts TCP performance.

The other work we reviewed was Performance modeling and analysis of TCP and UDP flows over software defined networks [5]. From this paper we were able to understand that SDN has three cooperative planes: data plane, control plane and application plane. The data plane handles the packet forwarding and switching whereas the control plane handles the routing.This separation supports automation, virtualization and dynamic traffic engineering.

Simulation Design

The researchers utilized ns-3 [6], which is a discrete-event network simulator where time advances as specific events are scheduled and executed. Unlike Cisco Packet Tracer, which focuses on emulating physical devices, ns-3 provides a programmable environment for highly repeatable research scenarios.

All simulation cases employed a dumbbell topology because it is simple to configure and is the standard for congestion control analysis. This setup forces all traffic through a single bottleneck link to observe protocol behavior under stress. As illustrated in Figure 2, source nodes are positioned on the left and destination nodes are on the right, with two routers acting as the central bridge. The edge nodes connect to these routers via 100Mbps standard fast ethernet links with a 1ms delay. The central bottleneck link between the routers operates at 10Mbps with a 10ms delay, which effectively simulates an access link like a cell tower. Each simulation ran for a duration of 20 seconds.

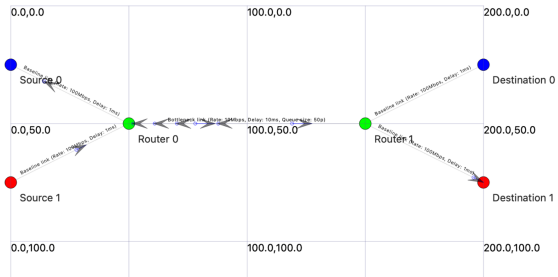

Figure 2: Dumbbell topology

For data collection and verification, the researchers used AnimationInterface [8] and NetAnim [9] to generate visual representations of the topology. FlowMonitor [10] tracked packet flows and aggregated performance metrics across all hosts. Additionally, pcap [11] files were exported and analyzed via Wireshark [12] to identify specific TCP behaviors, including suspected fast retransmissions, retransmissions, and duplicate ACKs.

The analysis focused on several key metrics to evaluate performance. Mean Delay measures the average travel time from source to destination, while the Packet Loss Ratio (%) represents the percentage of packets lost during transit. Throughput (Mbps) was calculated by multiplying the TX bitrate (Mbps) [A] by the percentage of successfully delivered packets.

To delve deeper into efficiency, the researchers calculated the TCP suspected fast retransmission ratio (%), which identifies packets re-sent after three duplicate ACKs, the suspected retransmission ratio (%) for packets re-sent after a timeout, and the TCP duplicate ACK ratio (%) for packets the indicate missing packets. These metrics were derived by dividing the raw counts by the product of received packets [B] and the number of TCP source nodes. Finally, Jain's Fairness Index [13] was used to determine how equitably resources were shared. A value of 1 indicates perfect fairness, while a value of 0.5 suggests significant imbalance between the two protocols. Throughput values were normalized against their intended network share before being processed through the fairness formula.

Simulations are performed across three different class modes, namely TCP only, UDP only, and mixed TCP and UDP. In the mixed mode, it simulates a real world scenario where TCP consumes 80% of the total traffic and the remaining 20% is used by UDP.

Experimental Setup

The researchers conducted three distinct simulations to fulfill the requirements of this study.

Buffer simulation was inspired by the optical packet-switched network's low buffer size. The goal is to analyze how memory or buffer size of a router affects data flow in a standard electrical network. The key parameter is the source router buffer size, tested against the values 2, 10, 50 (baseline), 100, and 500. All applicable metrics are gathered and simulations are carried out across all modes. Note that in UDP mode, TCP retransmissions and TCP duplicate ACKs are not measured. Jain's Fairness Index is not measured in non mixed mode.

The class imbalance simulation was inspired by weighted fair queueing mechanisms in optical networks and dynamic reprioritization in Software Defined Networking (SDN) used to resolve class unfairness. The goal was to evaluate how TCP and UDP flows compete for bandwidth in a standard network environment. The key parameter for this experiment was the TCP:UDP ratio with tested values of 9:1, 7:3, 5:5 (baseline), 3:1, and 1:9. This simulation was performed exclusively in mixed mode. Building on findings from the previous experiment which showed combined throughput equaled the bottleneck capacity, the researchers adjusted the UDP data rate to 1.1 Mbps. This change ensured UDP grew faster than TCP so that nine sources would total 9.9 Mbps and leave only 0.1 Mbps for the TCP flow.

The latency simulation was inspired by the reliability of optical packet-switched networks

over long fiber links which is a characteristic not inherently found in standard ethernet links. The objective was to analyze how latency and interference affect both TCP and UDP flows. The researchers tested latency values of 10ms (baseline), 50ms, 100ms, 200ms, and 400ms across all simulation modes. To reflect real-world scenarios, a Bit Error Rate of 0.0001% was introduced to simulate packet loss from interference. In earlier iterations, throughput and fairness remained consistent which the researchers hypothesized was due to a short duration preventing the network from reaching a steady state. They suspected the initial traffic burst overshadowed the latency effects typically managed by TCP congestion control. Another potential factor was that classes were originally separated by host which allowed the router to throttle them individually. Consequently, the researchers increased the simulation duration to 40 seconds and configured the setup so that TCP and UDP packets originated from the same source.

If you are interested in the entire process we followed, please visit the project GitHub repository [1].

Results

In the buffer simulation, the researchers observed that throughput and fairness remained constant regardless of the buffer size. While it is typically expected that a smaller buffer reduces throughput, the team deduced that the experimental setup was flawed because the combined throughput consistently equaled the bottleneck link capacity. This indicates that the network reached high utilization but did not experience actual congestion. However, TCP mean delay and packet loss were notably affected by the buffer size. Figure 3 illustrates that mean delay increases alongside buffer size, while Figure 4 shows that packet loss increases as buffer size decreases. This occurs because larger buffers hold packets in the queue for longer periods, which results in fewer dropped packets. Conversely, small buffers cannot accommodate packet accumulation, leading to higher drop rates.

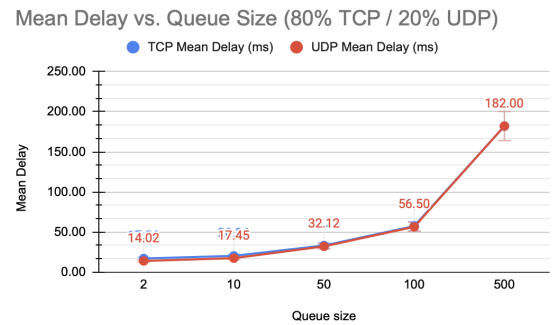

Figure 3: Mean delay vs queue size

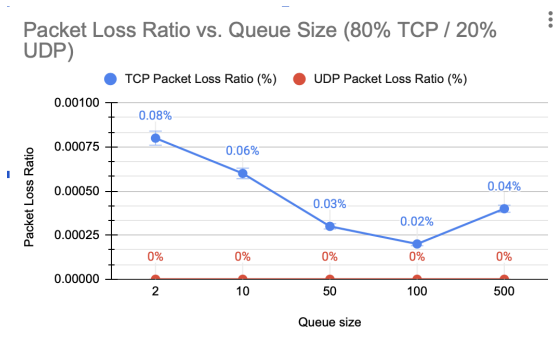

Figure 4: Packet loss ratio vs queue size

As shown in Figures 5, 6, and 7 decreasing the buffer size causes an increase in TCP retransmissions and duplicate ACKs. Since packet loss is inversely proportional to buffer size and TCP requires the delivery of an entire stream, these losses necessitate retransmissions. Consequently, one can deduce that the goodput, or the actual useful data received by destination nodes, decreases under these conditions.

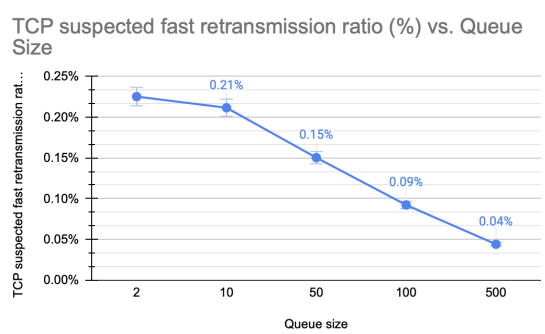

Figure 5: TCP suspected fast retransmission ratio (%) vs queue size

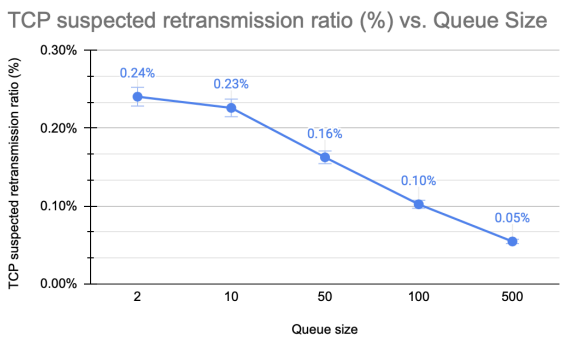

Figure 6: TCP suspected retransmission ratio (%) vs queue size

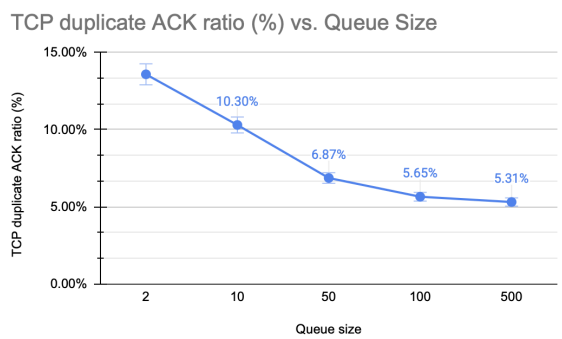

Figure 7: TCP duplicate ACK ratio (%) vs queue size

In the class imbalance simulation, the researchers found results similar to the buffer study where throughput and fairness remained unchanged regardless of the class balance, even though the team expected UDP to starve TCP. The results showed that TCP retransmissions and duplicate ACKs also increased with the UDP share, like in the previous simulation. One hypothesis, based on NetAnim animations, is that the slow start mechanism remains active for most of the experiment. This initial burst of TCP packets appears to amortize the throughput, preventing the network from reaching a UDP starvation state. Another possibility is that because TCP and UDP are sent through different links, the router utilizes a Flow Queue Controlled Delay [14] mechanism to distribute them equally at the egress link. While other metrics provided less insight, mean delay and packet loss increased with the UDP share as seen in Figures 8 and 9. This suggests the onset of congestion collapse where UDP is just beginning to starve the TCP flows.

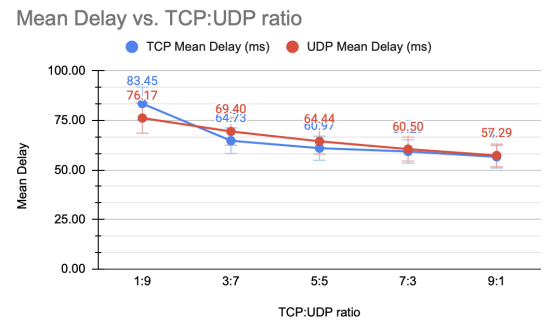

Figure 8: Mean delay vs TCP:UDP ratio

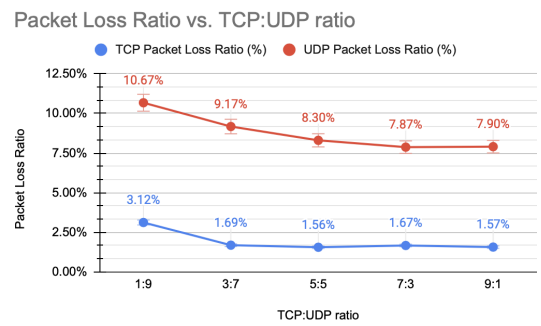

Figure 9: Packet loss ratio vs TCP:UDP ratio

The latency simulation results in Figures 10 and 11 demonstrate that TCP throughput and fairness dip as delay increases. This occurs because the congestion window is constrained by both latency and the error rate. TCP congestion signals are triggered when packets are lost randomly or when ACKs take longer to return, which prompts the sender to decrease the TCP rate while the UDP rate remains constant.

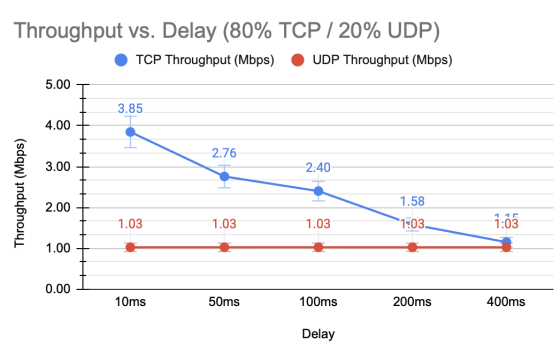

Figure 10: Throughput (Mbps) vs delay

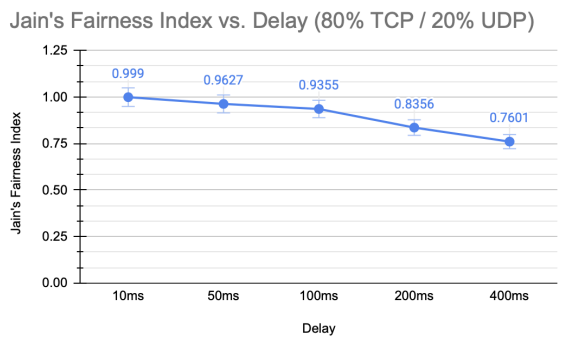

Figure 11: Jain's Fairness Index vs delay

Figure 12 indicates that TCP duplicate ACKs increase as delay rises. In low-latency networks, packet losses are rectified almost instantaneously, but in high-latency scenarios, the sender detects the loss later, which causes packets to be sent out of order. Interestingly, TCP retransmissions actually decrease as delay increases. While this seems counterintuitive, retransmissions do not rise with duplicate ACKs because the congestion window shrinks, meaning the sender transmits fewer packets to adjust for the congestion and prevent timeouts.

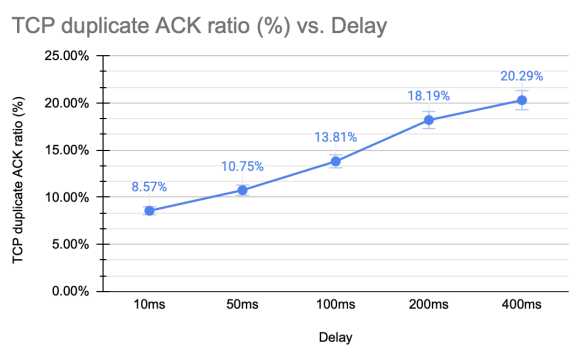

Figure 12: TCP duplicate ACK ratio vs delay

Across all simulations, the researchers found that UDP packet loss is significantly higher than TCP packet loss. This disparity stems from the lack of reliability in UDP, whereas TCP utilizes flow and congestion control to ensure efficient network use. UDP lacks handshake and recovery mechanisms because it is a fire-and-forget protocol. As UDP bandwidth usage increases, its loss rate grows drastically compared to TCP. At 1 Mbps, the UDP loss rate is 3.5 times higher than TCP, but at 10 Mbps, it becomes 1000 times higher, reaching a staggering 45% packet loss rate as shown in Figure 13.

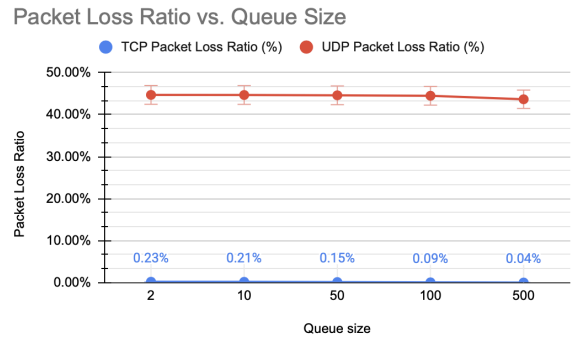

Figure 13: Packet loss ratio vs queue size

Discussion

From the results of the buffer simulation, we can deduce that routers do not require extensive buffering as long as the data flow remains within the network constraints. This aligns with findings in optical packet-switched (OPS) networks which are limited to very small buffer sizes of often only one or two packets. Instead of dropping packets due to buffer overflow, OPS networks utilize deflection routing where packets are redirected to longer links to buy time. In traditional electronic networks, a similar effect is achieved by increasing link bandwidth which has been a primary method of improvement since the inception of computer networking. While electronic networks can balance mean delay and packet loss by tweaking buffer sizes, OPS networks lack this flexibility because large optical buffers are prohibitively expensive due to a lack of optical memory.

Furthermore, buffers in OPS networks introduce significant overhead because light must be converted to electrical signals and then back to light before transmission. To maintain optimal speed and reliability with small buffers, OPS networks prioritize deflection routing. A key difference found in the related work is that while electronic networks reduce losses by increasing buffer sizes, optical switched networks primarily

reduce loss through packet aggregation where multiple packets are bundled into a single optical unit. This matches our finding that throughput remains stable if the pipe is full, but the mechanism for efficiency differs between the two technologies.

In the latency simulation, the observed dip in throughput and fairness matches the findings in studies of optical packet-switched networks. In those studies, researchers noted that while packet aggregation decreases loss, it inherently increases delay and round-trip time. This increased latency triggers TCP congestion control mechanisms and causes senders to decrease throughput to avoid perceived congestion. This behavior is also reflected in the survey on transport layer protocols which notes that TCP is highly sensitive to the timing of acknowledgments whereas UDP remains indifferent to such delays.

Our experimental results show that UDP packet loss is significantly higher than TCP packet loss which lines up with what other researchers have found. In studies of standard wired networks TCP often hits near 0% packet loss because it uses connection oriented flow control to stay reliable. In those same studies UDP loss can skyrocket to over 95% when bandwidth is restricted. The simulation we ran for this project followed this exact trend even though our maximum loss topped out at 45% during a 10 Mbps bottleneck. The gap between our 45% loss and the higher figures in other papers comes down to how much data was being sent versus the capacity of the bottleneck rather than any special strategies to stop the loss. Research into Software Defined Networking suggests that certain network setups can actually make UDP performance even worse. Since UDP is connectionless every initial packet has to be redirected to a controller before a rule is set up which creates a bottleneck in the control plane. This can increase packet loss by a huge margin compared to TCP which only has to redirect its very first handshake packet.

Ultimately the dumbbell topology we used in this simulation highlights how buffer overflows, class imbalance, and latency causes problems. The main takeaway from the research is that UDP lacks the congestion control found at Layer 4 which leads to that classic fire and forget behavior. This makes the protocol extremely vulnerable whenever a link gets saturated or the architecture adds too much overhead.

Conclusion

TCP and UDP are major transport layer protocols, each offering unique advantages.

TCP ensures reliability, while UDP delivers speed, making the choice dependent on the specific needs of the data transfer. The study has shown us that TCP reduces throughput under congestion and UDP maintains a higher rate. TCP treats packet loss or delay as a sign of overload. It cuts its congestion window often by half and retransmits lost packets. This slows the sender to keep the network stable and fair. On the other hand, UDP does not have congestion control. It keeps sending data at the same rate regardless of network load. UDP is designed for real time and low latency applications where speed matters more than reliability. As a result UPD can appear to maintain a higher rate, even though more packets may be lost. TCP should be used where reliability is required such as downloads and file transfer whereas UDP should be used for broadcasting like gaming and video calling.

TCP guarantees delivery but pays the price in higher latency, retransmissions, and reduced throughput when congestion occurs, while UDP stays fast and persistent because it ignores congestion and reliability entirely. To keep the network fair and stable, routers do the heavy lifting through queue management and scheduling. Queue management helps to avoid congestion and scheduling ensures flows to get fair and weights access to bandwidth. The network does not need a high buffer if there is a workaround. Active queue management keeps

the queue short, fair scheduling prevents on flow from hopping the buffer and ECN signals congestion without dropping packets.

Definition of Terms

[A] TX bitrate (Transmission bitrate) - The speed at which data is sent out.
[B] rxPackets - The total number of packets successfully received.

References

[1] GitHub repository <https://github.com/ianbunag/cmpe-206-computer-network-design-project>
[2] Survey on Transport Layer Protocols: TCP & UDP <https://www.ijcaonline.org/archives/volume46/number7/6920-9285/>
[3] Performance Comparison between TCP and UDP Protocols in Different Simulation Scenarios <https://www.academia.edu/37966449/Performance_Comparison_between_TCP_and_UDP_Protocols_in_Different_Simulation_Scenarios>
[4] TCP and UDP performance for Internet over optical packet-switched networks <https://www.sciencedirect.com/science/article/abs/pii/S1389128604000593>
[5] Performance modeling and analysis of TCP and UDP flows over software defined networks <https://www.sciencedirect.com/science/article/abs/pii/S1084804519300189>
[6] ns-3 Network Simulator <https://www.nsnam.org/>
[7] Cisco Packet Tracer <https://www.netacad.com/cisco-packet-tracer>
[8] ns3::AnimationInterface Class Reference <https://www.nsnam.org/docs/release/3.18/doxygen/classns3_1_1_animation_interface.html>
[9] NetAnim <https://www.nsnam.org/wiki/NetAnim>
[10] FlowMonitor <https://www.nsnam.org/docs/models/html/flow-monitor.html>
[11] Pcap <https://en.wikipedia.org/wiki/Pcap>
[12] Wireshark <https://www.wireshark.org/>
[13] Jain's Fairness Index <https://en.wikipedia.org/wiki/Fairness_measure>
[14] Flow Queue Controlled Delay <https://en.wikipedia.org/wiki/CoDel>